

HTML编辑器架构设计文档

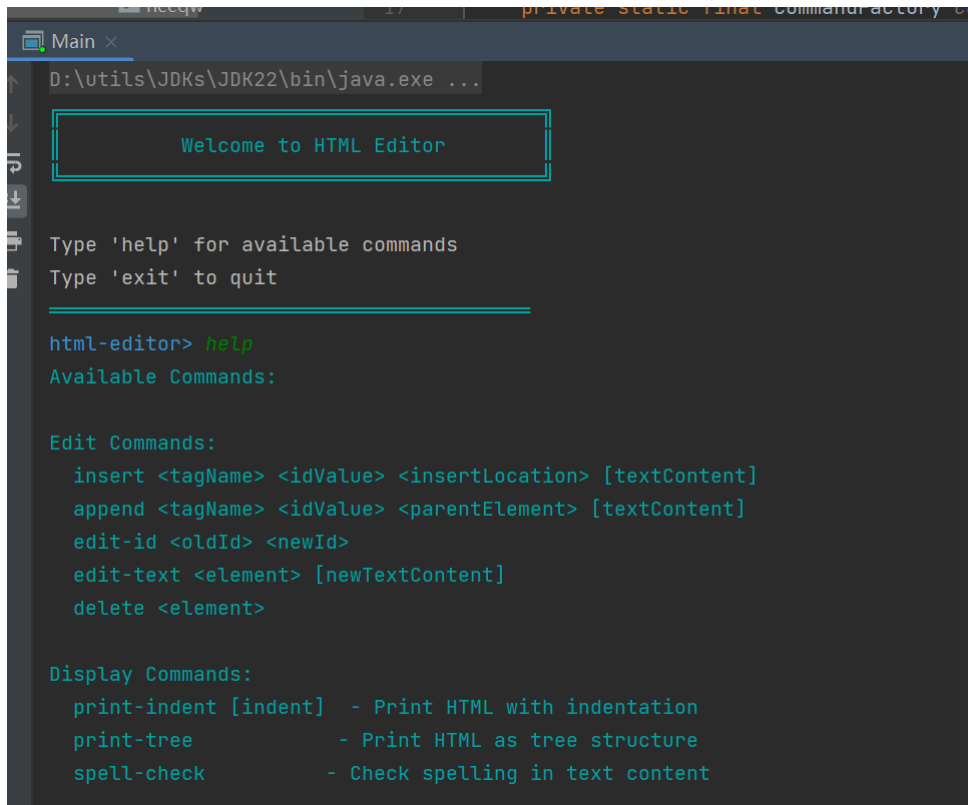
1. 系统概述

该HTML编辑器系统基于命令模式，模块化设计使其具备良好的可扩展性和灵活性。通过设计模式和SOLID原则的应用，系统实现了撤销/重做功能，同时保持操作的一致性和模块之间的低耦合性。

2. 运行方式

2.1 运行程序

1. 使用idea打开项目(html-editor)
2. 通过maven加载配置依赖(pom.xml)
3. 切换项目结构至**production**，运行Main.java中的 `main` 函数，**html-editor**>后输入命令，help可获得命令使用提示



```
D:\utils\JDKs\JDK22\bin\java.exe ...

Welcome to HTML Editor

Type 'help' for available commands
Type 'exit' to quit

html-editor> help
Available Commands:

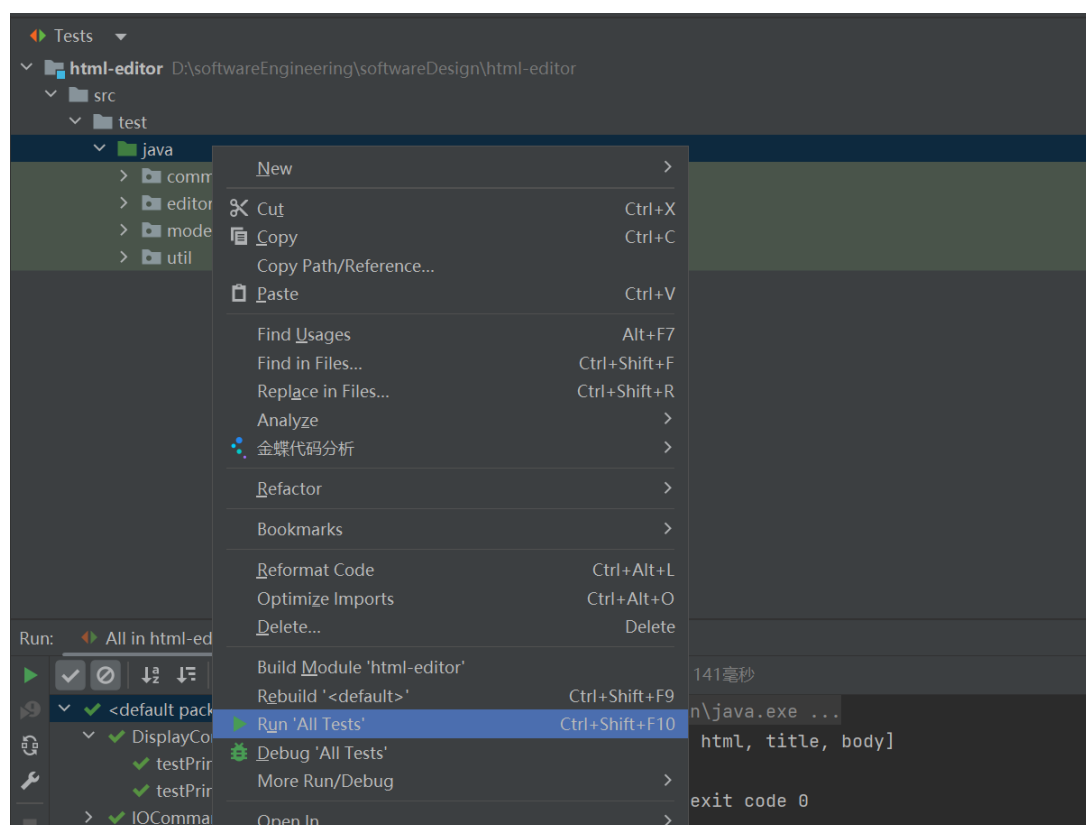
Edit Commands:
  insert <tagName> <idValue> <insertLocation> [textContent]
  append <tagName> <idValue> <parentElement> [textContent]
  edit-id <oldId> <newId>
  edit-text <element> [newTextContent]
  delete <element>

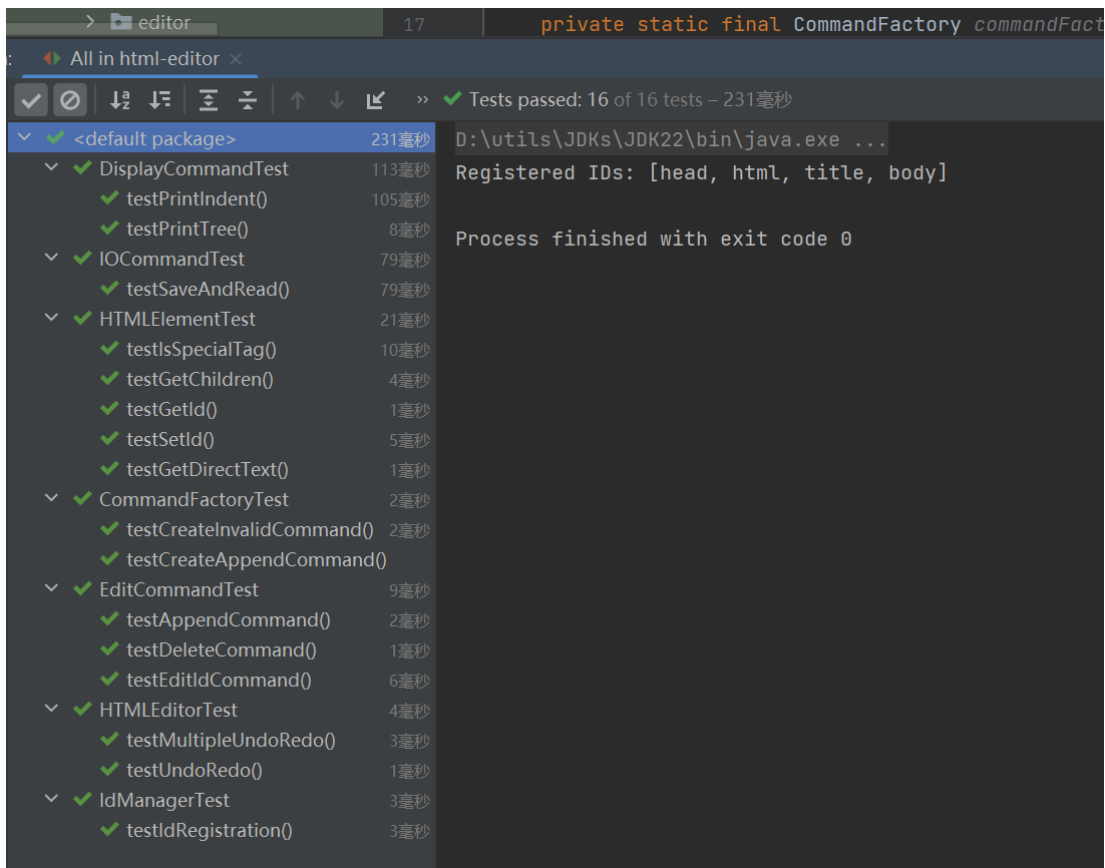
Display Commands:
  print-indent [indent] - Print HTML with indentation
  print-tree           - Print HTML as tree structure
  spell-check          - Check spelling in text content
```

```
html-editor> init
Registered IDs: [head, html, title, body]
✓ Command executed successfully.
html-editor> print-indent
<#root id="">
  <html>
    <head>
      <title></title>
    </head>
    <body></body>
  </html>
</#root>
✓ Command executed successfully.
html-editor> print-tree
└─ #root#
   └─ html
      ├── head
      │   └─ title
      └─ body
✓ Command executed successfully.
```

2.2 运行自动化测试

在Maven项目中切换项目结构至`test`，选中test根文件夹右键执行 `run 'all tests'` 即可实现自动化测试





3. 核心架构

系统划分为以下几个模块：

- **编辑器核心 (editor)**：负责管理文档状态和执行命令。
- **模型层 (model)**：定义HTML文档的数据结构。
- **命令系统 (command)**：包含所有可执行的操作和编辑命令。
- **工具类 (util)**：提供辅助支持功能。

3.1 实现结构及规则约束

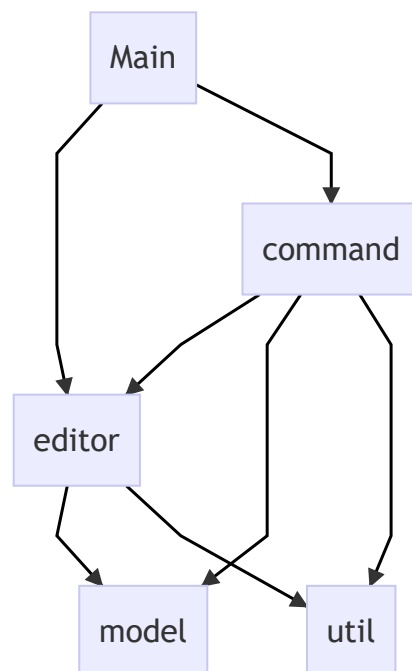
代码结构如下

```
| Main.java # 程序入口，处理命令行交互
|
|---command # 命令系统根目录
| | Command.java # 命令接口，定义execute()和getType()方法
| | CommandFactory.java # 命令工厂，负责创建具体命令实例
| | CommandType.java # 命令类型枚举(EDIT, DISPLAY, IO, CONTROL)
| |
| |---ControlCommand # 控制类命令目录
| | | RedoCommand.java # 重做命令实现
| | | UndoCommand.java # 撤销命令实现
| |
| |---DisplayCommand # 显示类命令目录
| | | DisplayCommand.java # 显示命令抽象基类
| | | PrintIndentCommand.java # 缩进格式显示命令
| | | PrintTreeCommand.java # 树形格式显示命令
| | | SpellCheckCommand.java # 拼写检查命令
| |
| |---EditCommand # 编辑类命令目录
| | | AppendCommand.java # 添加子元素命令
```

		DeleteCommand.java	# 删除元素命令
		EditCommand.java	# 编辑命令抽象基类
		EditIdCommand.java	# 修改元素ID命令
		EditTextCommand.java	# 修改元素文本命令
		InsertCommand.java	# 插入元素命令
		└IOCommand	# 输入输出命令目录
		InitCommand.java	# 初始化编辑器命令
		IOCommand.java	# IO命令抽象基类
		ReadCommand.java	# 读取HTML文件命令
		SaveCommand.java	# 保存HTML文件命令
		└editor	# 编辑器核心目录
		DocumentState.java	# 文档状态类，用于状态保存和恢复
		HTMLEditor.java	# HTML编辑器核心类，管理文档状态和命令执行
		└model	# 数据模型目录
		HTMLElement.java	# HTML元素类，封装DOM操作
		└util	# 工具类目录
		CommandParser.java	# 命令解析器，处理命令行输入
		ConsoleColors.java	# 控制台颜色工具类
		IdManager.java	# ID管理器，维护ID唯一性

3.2 模块依赖关系

以下是模块之间的依赖关系图：



4. 模块详细描述

4.1 编辑器核心 (editor)

- **HTMLEditor**: 应用**单例模式**, 确保全局唯一的编辑器实例。负责维护文档状态、命令执行和撤销/重做管理。
- **DocumentState**: 使用**备忘录模式**保存文档的完整状态, 包括DOM结构和ID注册信息, 支持状态保存和恢复操作。

4.2 模型层 (model)

- **HTMLElement**: 封装 `jsoup` 的 `Element` 类, 为HTML元素提供高级接口, 允许进行DOM操作、ID管理和文本内容操作。

4.3 命令系统 (command)

包含多种命令类型, 支持对文档的各种操作:

1. 编辑命令 (EditCommand):

- `AppendCommand`: 添加子元素。
- `InsertCommand`: 插入元素。
- `DeleteCommand`: 删除元素。
- `EditIdCommand`: 修改ID。
- `EditTextCommand`: 修改文本内容。

2. 显示命令 (DisplayCommand):

- `PrintTreeCommand`: 树形显示。
- `PrintIndentCommand`: 缩进显示。
- `SpellCheckCommand`: 拼写检查。

3. IO命令 (IOCommand):

- `InitCommand`: 初始化文档。
- `ReadCommand`: 读取文件内容。
- `SaveCommand`: 保存文档内容。

4. 控制命令 (ControlCommand):

- `UndoCommand`: 撤销上一步操作。
- `RedoCommand`: 重做被撤销的操作。

4.4 工具类 (util)

- **IdManager**: 管理HTML元素ID的唯一性, 确保特殊标签的ID不重复, 维护ID注册表。
- **CommandParser**: 解析命令行输入, 处理命令参数, 支持带引号的参数解析。

5. 设计模式应用

5.1 命令模式 (Command Pattern)

每个操作都被封装为一个独立的命令对象，支持撤销/重做，并实现调用者与接收者的分离。

5.2 工厂模式 (Factory Pattern)

通过 `CommandFactory` 类来统一创建命令对象，简化客户端调用并封装创建逻辑。

5.3 单例模式 (Singleton Pattern)

`HTMLoader` 和 `IdManager` 类使用单例模式，确保只有一个全局实例。

5.4 备忘录模式 (Memento Pattern)

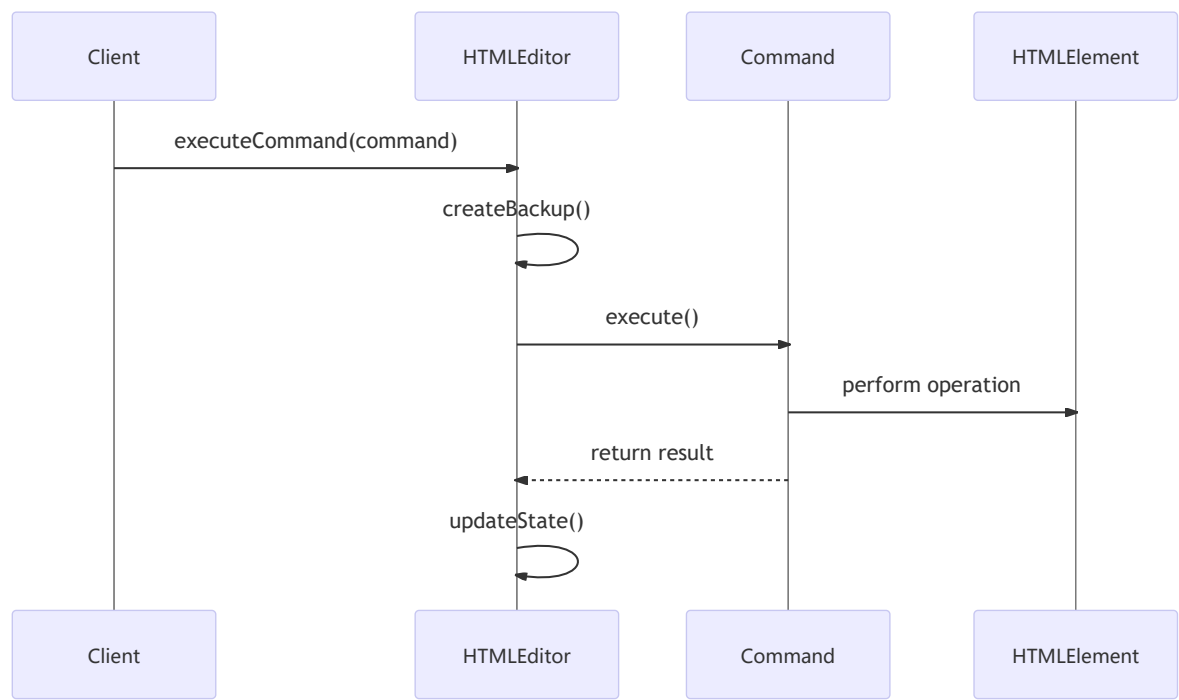
`DocumentState` 类负责保存文档状态，允许随时回滚到之前的状态，确保操作的一致性。

5.5 装饰器模式 (Decorator Pattern)

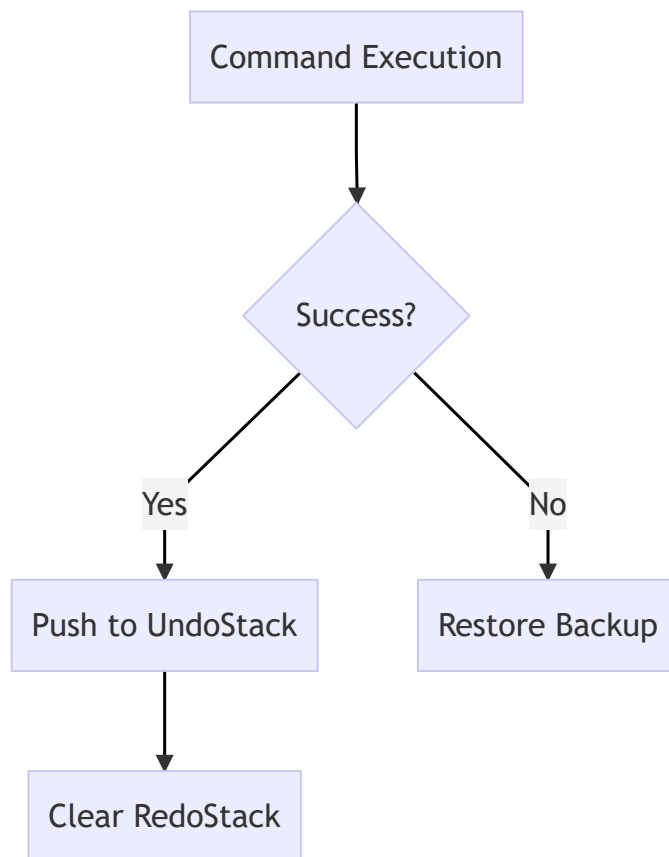
通过扩展 `HTMLElement` 类来实现对DOM操作的额外支持，同时保留与 `jsoup` 库的接口一致性。

6. 关键设计决策

6.1 命令执行流程



6.2 状态管理策略



- **备忘录模式**：保存和恢复文档状态，便于撤销操作。
- **集中管理**：所有状态变更统一由 `HTML editor` 管理。
- **原子性**：命令执行过程中确保操作原子性，出现异常时可回滚。

6.3 封装第三方依赖

通过将 `jsoup` 封装在 `HTML element` 类中，系统提供统一的DOM操作接口，降低对外部依赖的耦合性，便于未来更换底层库。

6.4 错误处理机制

- **异常处理层次**:
 1. 命令级别验证
 2. 编辑器级别状态管理
 3. 全局错误恢复
- **状态一致性保证**:
 1. 操作前备份
 2. 原子性执行
 3. 失败时回滚

7. 运行和测试方法

7.1 运行

1. 初始化编辑器环境并创建 `HTMLEditor` 实例。
2. 使用 `CommandParser` 解析命令输入，将其转化为相应的命令对象。
3. 执行命令对象的操作，通过 `HTMLEditor` 更新状态并维护命令历史。

7.2 测试策略

单元测试

- **命令执行测试**：针对不同类型命令，验证其是否正确执行。
- **状态管理测试**：确保撤销/重做和状态恢复功能正常。
- **工具类测试**：测试 `IdManager` 和 `CommandParser` 的核心功能。

集成测试

- **命令序列测试**：多种命令连续执行，验证系统是否保持一致性。
- **撤销/重做链测试**：多次撤销和重做，确保状态正确。
- **状态一致性测试**：在各种操作下测试文档状态的一致性。

8. 扩展性和性能

8.1 扩展性

- **新命令添加**：只需实现 `Command` 接口并注册至 `CommandFactory`，无需修改现有代码。
- **新功能集成**：通过扩展 `HTML_Element` 或新增工具类实现。

8.2 性能考虑

- **内存优化**：优化状态备份，控制命令栈大小，节省内存。